Design and Modeling of Complex Software Architectures

Lab Assignment 02, Documentation & Requirements

Application Scenario 4: Instant Mobility Platform

Team: The Winx | Summer Semester 2026

1 PLATFORM OVERVIEW

The Instant Mobility Platform aggregates the offerings of multiple vehicle-sharing providers into a single application. Instead of installing one app per provider, users can search for any type of available vehicle (e-scooter, bicycle, e-bike, electric car) near their current location, book it instantly, ride, and pay, all in one place.

Providers use the platform to register their fleets, configure pricing and usage restrictions per vehicle, and monitor real-time fleet availability and location.

The system is built as a microservice architecture using Java/Kotlin and the Spring framework, with each domain boundary corresponding to an independent deployable service.

2 FUNCTIONAL REQUIREMENTS

2.1 ACTORS AND ROLES

Actor	Description	Responsibilities
User (Customer)	A registered person who uses the platform to rent vehicles.	Register, log in, search & filter vehicles, book, end ride, pay, rate, view history.
Provider	A company or individual who lists vehicles for rent on the platform.	Register, log in, manage fleet (CRUD), set pricing & restrictions, monitor fleet status.
System	Internal platform processes that run without direct user interaction.	Update vehicle status on booking events, compute ride cost, trigger payment processing.

2.2 REQUIREMENT CATALOGUE

ID	Name	Description	Priority	Actors
USER SIDE				
R01	**User Registration**	A new customer registers by providing their full name, email address, password, phone number, and date of birth. The account is required to access all booking and payment features on the platform.	**High**	User
R02	**User Login**	A registered user authenticates with their email and password. After successful login a session is established that grants access to booking, payment, and history features.	**High**	User

R03	**Search Vehicles by Location**	A user can search for available vehicles near their current geographic coordinates. The platform returns all nearby vehicles whose status is AVAILABLE, sorted by proximity.	**High**	User
R04	**Filter Vehicles**	A user can apply one or more filters to a search result set. Supported filters include vehicle type (e-scooter, bicycle, e-bike, e-car), price range, minimum age requirement, and maximum trip duration.	**Medium**	User
R05	**Book Vehicle**	A user can immediately book an available vehicle. Upon booking the vehicle status is set to BOOKED, its start location and time are recorded, and the vehicle is no longer bookable by other users.	**High**	User
R06	**End Booking**	A user can terminate their active booking through the platform. The system records the end time and GPS coordinates, computes the total distance and cost based on the vehicle's billing model, and sets the booking status to COMPLETED.	**High**	User, System
R07	**Process Payment**	After a booking is completed, the platform calculates the final charge and processes the payment automatically. A Payment record is created with the amount, payment method, and status (PENDING → PAID or FAILED).	**High**	User, System
R08	**Rate Vehicle and Provider**	After a booking reaches COMPLETED status, the user can submit a rating for the vehicle and the provider independently, each scored 1–5, with an optional text comment. Only one rating per booking is permitted.	**Low**	User

R09	**View Booking History**	A user can view a chronological list of all their past bookings. Each entry displays the vehicle type, start and end times, distance, total cost, and current booking status.	**Medium**	User
PROVIDER SIDE				
R10	**Provider Registration**	A company or individual registers as a provider by supplying company name, contact name, email address, password, and phone number. A provider account is required to list vehicles on the platform.	**High**	Provider
R11	**Provider Login**	A registered provider authenticates with their email and password. The session grants access to vehicle management and fleet monitoring features.	**High**	Provider
R12	**Manage Vehicles (CRUD)**	A provider can create new vehicles specifying type, description, pricing, and restrictions. They can view, update, and delete their own vehicles at any time. A vehicle cannot be deleted while a booking is ACTIVE.	**High**	Provider
R13	**Define Pricing Model**	For each vehicle a provider sets a unit price and selects a billing model: PER_HOUR (charged per started hour) or PER_KILOMETER (charged per kilometre travelled). The total cost is computed automatically when a booking ends.	**High**	Provider
R14	**Define Vehicle Restrictions**	A provider can optionally set per-vehicle restrictions: maximum booking duration (minutes), maximum distance (km), minimum driver age, and maximum number of persons. The platform enforces	**Medium**	Provider

		these restrictions during booking validation.		
R15	**View Fleet Status**	A provider can view a real-time overview of all their vehicles, including each vehicle's current GPS coordinates and availability status (AVAILABLE or BOOKED). This allows providers to monitor their fleet at a glance.	**Medium**	Provider

3 DOMAIN MODEL

The domain model describes the core business objects (entities), their attributes, and the relationships between them. Location data and vehicle restrictions are embedded as attributes on the Vehicle and Booking classes rather than as separate entities, keeping the model focused and avoiding unnecessary join complexity.

3.1 CORE ENTITIES

Entity	Purpose
User	Represents a customer who uses the platform to search, book, pay for, and rate vehicles. A User can have many bookings and can write multiple ratings.
Provider	Represents a company or individual that lists vehicles for rent. A Provider owns a fleet of vehicles and can monitor their availability and location.
Vehicle	Represents a rentable mobility asset owned by a Provider. Vehicles carry their own pricing model, current GPS position, availability status, and all applicable usage restrictions as embedded attributes.
Booking	Records a single rental transaction between a User and a Vehicle. Captures the full journey lifecycle: start location and time, end location and time, computed distance, total cost, and status transitions.

Payment	Records the financial transaction associated with a completed Booking. Its lifecycle is tightly coupled to the Booking, a Payment cannot exist independently. The status tracks the result of the payment processing.
Rating	Captures user feedback for both the Vehicle and Provider after a completed Booking. Exactly one Rating may be created per Booking. The scores for vehicle and provider are stored independently to allow separate analytics.

3.2 ENUMERATIONS

Enum	Used By	Values
`VehicleType`	Vehicle	`E_SCOOTER, BICYCLE, E_BIKE, E_CAR`
`VehicleStatus`	Vehicle	`AVAILABLE, BOOKED`
`BillingModel`	Vehicle	`PER_HOUR, PER_KILOMETER`
`BookingStatus`	Booking	`ACTIVE, COMPLETED, CANCELLED`
`PaymentStatus`	Payment	`PENDING, PAID, FAILED`

3.3 ENTITY ATTRIBUTES

User

Represents a customer who uses the platform to search, book, pay for, and rate vehicles. A User can have many bookings and can write multiple ratings.

Attribute	Type	Description	Key

id	`Long`	Unique identifier (primary key)	**PK**
name	`String`	Full name of the user	
email	`String`	Email address, used for login and contact	
passwordHash	`String`	Bcrypt-hashed password, never stored in plain text	
phoneNumber	`String`	Contact phone number	
dateOfBirth	`Date`	Used to validate age restrictions during booking	
registeredAt	`DateTime`	Timestamp of account creation	

Provider

Represents a company or individual that lists vehicles for rent. A Provider owns a fleet of vehicles and can monitor their availability and location.

Attribute	Type	Description	Key
id	`Long`	Unique identifier (primary key)	**PK**
companyName	`String`	Name of the provider company or individual	
contactName	`String`	Name of the responsible contact person	

email	`String`	Email address, used for login and contact	
passwordHash	`String`	Bcrypt-hashed password	
phoneNumber	`String`	Contact phone number	
registeredAt	`DateTime`	Timestamp of account creation	

Vehicle

Represents a rentable mobility asset owned by a Provider. Vehicles carry their own pricing model, current GPS position, availability status, and all applicable usage restrictions as embedded attributes.

Attribute	Type	Description	Key
id	`Long`	Unique identifier (primary key)	**PK**
providerId	`Long`	Foreign key to the owning Provider	**FK**
type	`VehicleType`	Enum: E_SCOOTER | BICYCLE | E_BIKE | E_CAR	
description	`String`	Free-text description (model, colour, features)	
status	`VehicleStatus`	Enum: AVAILABLE | BOOKED, updated on booking events	
currentLatitude	`Double`	Last known GPS latitude of the vehicle	

currentLongitude	Double	Last known GPS longitude of the vehicle	
pricePerUnit	BigDecimal	Price charged per unit (hour or kilometre)	
billingModel	BillingModel	Enum: PER_HOUR | PER_KILOMETER	
maxDurationMinutes	Integer	Maximum permitted booking duration (null = no limit)	
maxKilometers	Integer	Maximum permitted distance per booking (null = no limit)	
minAge	Integer	Minimum driver age in years (null = no restriction)	
maxPersons	Integer	Maximum number of persons allowed (null = no restriction)	

Booking

Records a single rental transaction between a User and a Vehicle. Captures the full journey lifecycle: start location and time, end location and time, computed distance, total cost, and status transitions.

Attribute	Type	Description	Key
id	Long	Unique identifier (primary key)	**PK**
userId	Long	Foreign key to the booking User	**FK**

vehicleId	Long	Foreign key to the booked Vehicle	**FK**
startTime	DateTime	Timestamp when the booking began	
endTime	DateTime	Timestamp when the booking ended (null while ACTIVE)	
startLatitude	Double	GPS latitude at booking start	
startLongitude	Double	GPS longitude at booking start	
endLatitude	Double	GPS latitude at booking end (null while ACTIVE)	
endLongitude	Double	GPS longitude at booking end (null while ACTIVE)	
distanceKm	Double	Total distance travelled in kilometres (computed on end)	
totalCost	BigDecimal	Final cost computed from pricePerUnit x billing model	
status	BookingStatus	Enum: ACTIVE | COMPLETED | CANCELLED	

Payment

Records the financial transaction associated with a completed Booking. Its lifecycle is tightly coupled to the Booking; a Payment cannot exist independently. The status tracks the result of the payment processing.

Attribute	Type	Description	Key
id	`Long`	Unique identifier (primary key)	**PK**
bookingId	`Long`	Foreign key to the associated Booking	**FK**
amount	`BigDecimal`	Charge amount (equals Booking.totalCost)	
paymentMethod	`String`	Payment method used (e.g. credit card, PayPal)	
paidAt	`DateTime`	Timestamp of successful payment (null if pending/failed)	
status	`PaymentStatus`	Enum: PENDING | PAID | FAILED	

Rating

Captures user feedback for both the Vehicle and Provider after a completed Booking. Exactly one Rating may be created per Booking. The scores for vehicle and provider are stored independently to allow separate analytics.

Attribute	Type	Description	Key
id	`Long`	Unique identifier (primary key)	**PK**
bookingId	`Long`	Foreign key to the rated Booking	**FK**
userId	`Long`	Foreign key to the User who wrote the rating	**FK**

vehicleId	Long	Foreign key to the rated Vehicle	**FK**
providerId	Long	Foreign key to the rated Provider	**FK**
vehicleScore	Integer	Vehicle score: 1 (poor) to 5 (excellent)	
providerScore	Integer	Provider score: 1 (poor) to 5 (excellent)	
comment	String	Optional free-text review comment	
createdAt	DateTime	Timestamp of rating submission	

3.4 RELATIONSHIPS

The table below documents every relationship in the domain model. Two relationship types from UML are used:

Composition (*--) the child entity cannot exist without the parent; its lifecycle is fully owned by the parent.

Association (-->) the two entities can exist independently; one simply references the other.

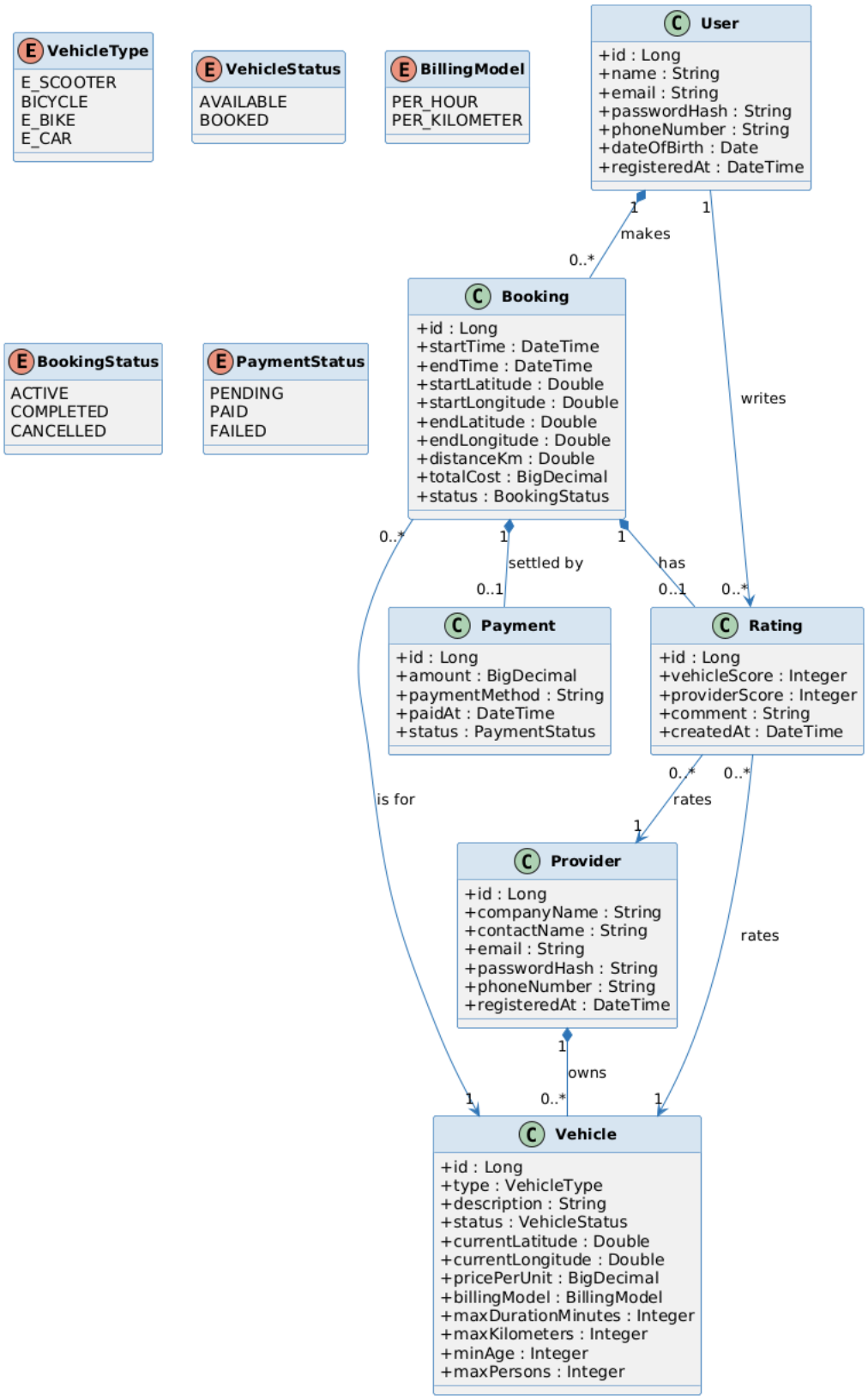
From	To	From mult.	To mult.	Type	Rationale
Provider	**Vehicle**	1	0..*	**Composition (*--)**	A Vehicle is exclusively owned by one Provider and cannot exist without it. Deleting a Provider cascades to their Vehicles.

User	**Booking**	1	0..*	**Composition (*--)**	A Booking is created by and permanently tied to a User. It cannot exist without the User.
Vehicle	**Booking**	1	0..*	Association (-->)	A Booking references the Vehicle being rented. The Vehicle exists independently, it has a life before and after the Booking.
Booking	**Payment**	1	0..1	**Composition (*--)**	A Payment is entirely owned by its Booking. It cannot exist without the Booking and is deleted with it.
Booking	**Rating**	1	0..1	**Composition (*--)**	A Rating is anchored to exactly one Booking. Only one rating is allowed per booking and it cannot exist without it.
User	**Rating**	1	0..*	Association (-->)	A User writes Ratings, but the Rating's lifecycle is owned by Booking (Composition above), not by the User.
Rating	**Vehicle**	0..*	1	Association (-->)	A Rating references the Vehicle that was rated. The Vehicle exists independently of any Rating.
Rating	**Provider**	0..*	1	Association (-->)	A Rating references the Provider that was rated. The Provider exists independently of any Rating.

3.5 PLANTUML SOURCE

The following PlantUML code produces the UML class diagram for the domain model. Paste it into plantuml.com or any PlantUML-compatible tool to render the diagram.

```
@startuml

' ==================== Enums ====================
enum VehicleType  { E_SCOOTER / BICYCLE / E_BIKE / E_CAR }
enum VehicleStatus { AVAILABLE / BOOKED }
enum BillingModel  { PER_HOUR / PER_KILOMETER }
enum BookingStatus { ACTIVE / COMPLETED / CANCELLED }
enum PaymentStatus { PENDING / PAID / FAILED }

' ==================== Classes ==================
class User     { +id:Long  +name:String  +email:String
                 +passwordHash:String  +phoneNumber:String
                 +dateOfBirth:Date  +registeredAt:DateTime }

class Provider { +id:Long  +companyName:String  +contactName:String
                 +email:String  +passwordHash:String
                 +phoneNumber:String  +registeredAt:DateTime }

class Vehicle  { +id:Long  +type:VehicleType  +description:String
                 +status:VehicleStatus
                 +currentLatitude:Double  +currentLongitude:Double
                 +pricePerUnit:BigDecimal  +billingModel:BillingModel
                 +maxDurationMinutes:Integer  +maxKilometers:Integer
                 +minAge:Integer  +maxPersons:Integer }

class Booking  { +id:Long  +startTime:DateTime  +endTime:DateTime
                 +startLatitude:Double  +startLongitude:Double
                 +endLatitude:Double  +endLongitude:Double
                 +distanceKm:Double  +totalCost:BigDecimal
                 +status:BookingStatus }

class Payment  { +id:Long  +amount:BigDecimal
                 +paymentMethod:String  +paidAt:DateTime
                 +status:PaymentStatus }

class Rating   { +id:Long  +vehicleScore:Integer  +providerScore:Integer
                 +comment:String  +createdAt:DateTime }

' ==================== Relationships =============
Provider "1"    *--  "0..*" Vehicle  : owns
User     "1"    *--  "0..*" Booking  : makes
Vehicle  "1"    <--  "0..*" Booking  : is for
Booking  "1"    *--  "0..1" Payment  : settled by
Booking  "1"    *--  "0..1" Rating   : has
User     "1"    -->  "0..*" Rating   : writes
Rating   "0..*" -->  "1"   Vehicle  : rates
Rating   "0..*" -->  "1"   Provider : rates

@enduml
```


VehicleType
E_SCOOTER
BICYCLE
E_BIKE
E_CAR
VehicleStatus
AVAILABLE
BOOKED
BillingModel
PER_HOUR
PER_KILOMETER
User
+id : Long
+name : String
+email : String
+passwordHash : String
+phoneNumber : String
+dateOfBirth : Date
+registeredAt : DateTime
1
makes
0..*
1
writes
Booking
+id : Long
+startTime : DateTime
+endTime : DateTime
+startLatitude : Double
+startLongitude : Double
+endLatitude : Double
+endLongitude : Double
+distanceKm : Double
+totalCost : BigDecimal
+status : BookingStatus
BookingStatus
ACTIVE
COMPLETED
CANCELLED
PaymentStatus
PENDING
PAID
FAILED
0..*
1
settled by
0..1
1
has
0..1
0..*
Payment
+id : Long
+amount : BigDecimal
+paymentMethod : String
+paidAt : DateTime
+status : PaymentStatus
Rating
+id : Long
+vehicleScore : Integer
+providerScore : Integer
+comment : String
+createdAt : DateTime
is for
0..*
0..*
rates
1
Provider
+id : Long
+companyName : String
+contactName : String
+email : String
+passwordHash : String
+phoneNumber : String
+registeredAt : DateTime
rates
1
owns
0..*
1
1
Vehicle
+id : Long
+type : VehicleType
+description : String
+status : VehicleStatus
+currentLatitude : Double
+currentLongitude : Double
+pricePerUnit : BigDecimal
+billingModel : BillingModel
+maxDurationMinutes : Integer
+maxKilometers : Integer
+minAge : Integer
+maxPersons : Integer